VERIQTA

SUBSCRIBER SERIES

REAL OUTAGE
REAL LESSONS
REAL IMPACT

# 10 KUBERNETES MISTAKES THAT TOOK DOWN OUR PRODUCTION SYSTEM

A DEEP DIVE INTO A REAL OUTAGE, WHAT WENT WRONG, HOW WE FOUND IT, AND HOW WE FIXED IT.


**IMPACT**
500K+ USERS AFFECTED

**DOWNTIME**
6 HOURS

**WHEN**
APR 15, 2024

**TARGET**
PRODUCTION ENVIRONMENT

## INSIDE THIS SERIES

- ✓ Timeline of the Outage
- ✓ 5 Critical Mistakes
- ✓ Investigation Journey
- ✓ The Architecture We Built
- ✓ Production Readiness Checklist
- ✓ 10 Lessons Learned

REAL INCIDENT — NOT HYPOTHETICAL
DEEP TECHNICAL — DEEP EXPLANATIONS
ACTIONABLE FIXES — PROVEN SOLUTIONS
PRODUCTION FOCUSED — BUILT FOR RELIABILITY

BUILT IN PRODUCTION FOR

*Every outage teaches a lesson. Every lesson builds a better system*

VERIQTA
Subscriber Series

☆ Subscriber Series ☆

# The Kubernetes Cluster That Went Down at 2 AM

## 5 Mistakes That Caused the Outage and How I Fixed Them

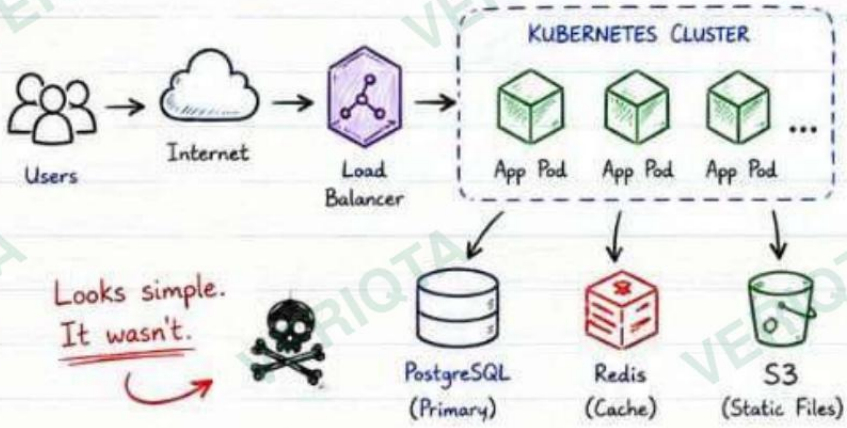
## 1. The Incident Overview

**2:07 AM**

- Production alerts started firing.
- Customer requests were timing out.
- Pods were restarting.
- CPU looked normal.
- Memory looked normal.

**YET THE APPLICATION WAS EFFECTIVELY UNAVAILABLE.**

OUR PRODUCTION ARCHITECTURE (SIMPLIFIED)




Looks simple. It wasn't.

### Timeline

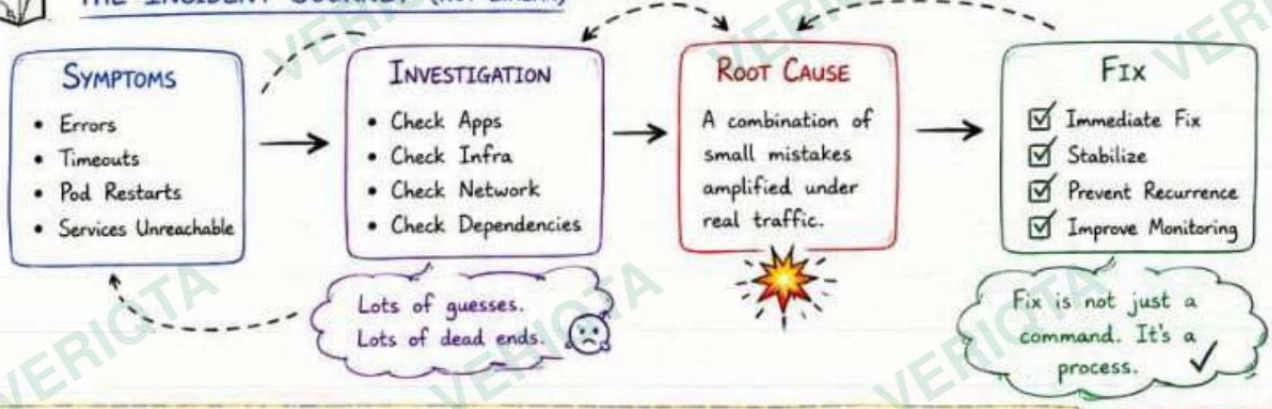
| | |
|---|---|
| 02:07 AM | Alerts triggered (High Error Rate) |
| 02:08 AM | On-call acknowledged |
| 02:10 AM | Customer complaints increased |
| 02:12 AM | Pods restarting detected |
| 02:18 AM | Initial checks (CPU, Memory) – Normal |
| 02:25 AM | Deeper investigation started |
| 02:47 AM | Root cause identified |
| 03:21 AM | Fix implemented |
| 03:48 AM | Traffic recovered |
| 04:05 AM | Incident resolved |

~ 1h 58m of pain

### Impact

| | |
|---|---|
| Users Affected | 18,000+ |
| Requests Failed | ~ 120K |
| Business Impact | High |
| Downtime | ~ 1h 58m |
| Customer Trust | ↓ |

### The Incident Journey (Not Linear)




### Why Incidents Are Rarely Caused by a Single Issue

Systems are complex. Components depend on each other. It's usually a chain reaction, not a single failure.

- → Small misconfigurations
- → Missing best practices
- → Assumptions that break under load
- → Gaps in monitoring
- → Lack of testing
- → Human error

☆ The goal is not to blame. The goal is to



REAL INCIDENTS. REAL LESSONS. STRONGER SYSTEMS. LET'S BUILD RESILIENT SYSTEMS TOGETHER!

The outage wasn't caused by one big bug. It was a combination of small mistakes.

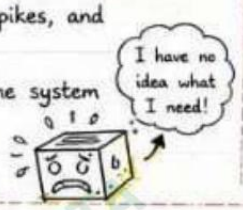
Root Cause #1

# Mistake #1: No Resource Requests

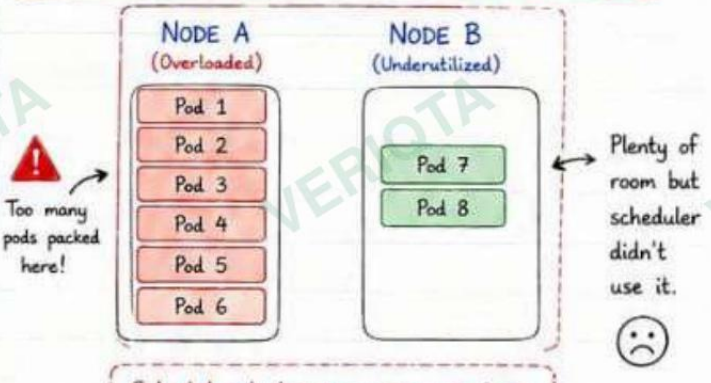
**NO REQUESTS = KUBERNETES IS GUESSING.**

## 1. What Went Wrong?

- Containers were deployed with NO CPU requests and NO Memory requests.
- The scheduler assumed nodes had more available resources than they actually did.
- When traffic spiked, multiple pods competed for the same CPU & Memory.
- This led to throttling, latency spikes, and pod restarts.
- Metrics looked "normal" until the system was already in pain.



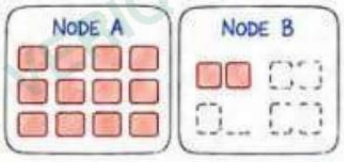

## 2. Node Scheduling Diagram (BAD EXAMPLE)



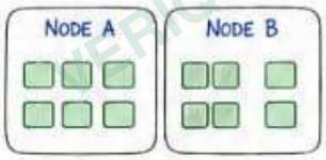

Scheduler lacks resource guarantees.

## 3. Pod Placement Comparison

| Without Requests ⊗ | With Requests ✓ |
|---|---|
| ⊗ Unpredictable scheduling | ✓ Predictable scheduling |
| ⊗ Resource starvation | ✓ Better resource balancing |
| ⊗ No guaranteed CPU | ✓ Guaranteed CPU |
| ⊗ No guaranteed Memory | ✓ Guaranteed Memory |
| ⊗ High latency | ✓ Stable performance |
| ⊗ Pods may get evicted | ✓ Happier users |




Unbalanced. Unreliable. Unsafe.




Balanced. Reliable. Scalable.

## 4. Bad Configuration Example

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - name: webapp
          image: nginx
```

X No CPU requests

X No Memory requests

## 5. Correct Configuration Example

```yaml
containers:
  - name: webapp
    image: nginx
    resources:
      requests:
        cpu: "250m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
```

Guaranteed minimum (used by Scheduler)

Maximum allowed (used by Kubelet)

## Interview Note

**Q: Why Are Requests Different From Limits?**

| Requests | VS | Limits |
|---|---|---|
| Used by Scheduler | | Used by Kubelet |
| ↓ | | ↓ |
| Determines where the pod can run | | Enforces maximum at runtime |
| ↓ | | ↓ |
| Guarantees minimum resources | | Prevents container from using too much |

Requests decide where a Pod runs.
Limits decide how far a Pod can go.

## 7. Troubleshooting Checklist

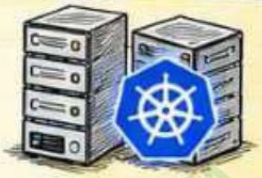
- ☐ Check Pod resource requests
- ☐ Check Pod resource limits
- ☐ Check Node utilization (CPU & Memory)
- ☐ Check for Pending Pods
- ☐ Review scheduler events
- ☐ Review OOMKilled events
- ☐ Compare requests vs actual usage

Always compare: kubectl top pods vs requests/limits

## 8. Key Lesson

The cluster wasn't actually out of CPU.
The scheduler simply had no reliable information to make good placement decisions.

Kubernetes performs best when you tell it what your workloads need.



**REMEMBER**

Good resource management is not just about performance. It's about stability.



**RESOURCE REQUESTS ARE NOT OPTIONAL DOCUMENTATION. → THEY ARE SCHEDULING INSTRUCTIONS.**

The app was healthy.
The cluster was healthy.
But traffic won.

# Mistake #3: No Horizontal Pod Autoscaler (HPA)

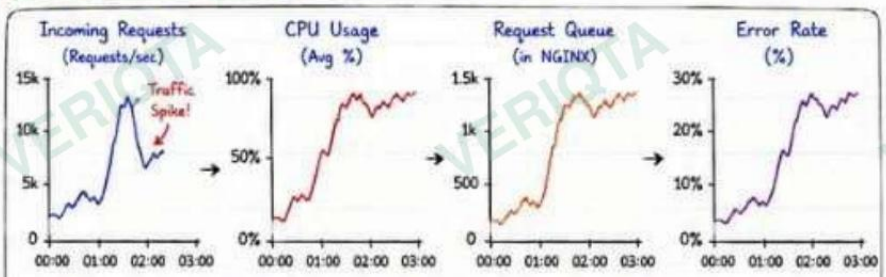
**⚠ TRAFFIC ARRIVED. CAPACITY DIDN'T.**

## 1. WHAT HAPPENED?

- Traffic suddenly spiked 10x.
- Our deployment had a fixed replica count (3).
- CPU usage shot up.
- Requests queued up.
- Users started timing out.
- No HPA = No automatic scaling.

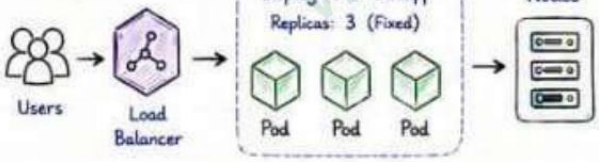
☹ Kubernetes can't scale what you never told it to scale.

## 2. THE TRAFFIC SPIKE



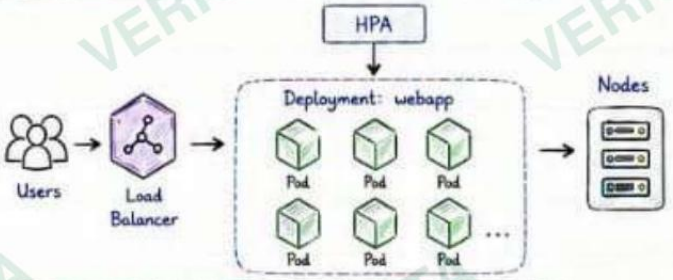

More traffic → More load → Queue builds → Timeouts → Outage ☹

## 3. WITHOUT HPA (WHAT WE HAD) ☹




⚠ No matter how much traffic comes, we only have 3 pods.
⚠ Pods get overloaded.
⚠ Response time goes up. Users time out.

## 4. WITH HPA (WHAT WE SHOULD HAVE HAD) ☺




✓ HPA watches CPU/Memory (or custom metrics).
✓ Automatically scales pods up and down.
✓ Traffic handled. Users happy!

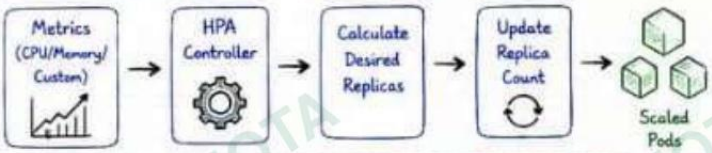
## 5. HPA CONFIG EXAMPLE ☺

HPA maintains balance between performance and cost.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 3
  maxReplicas: 15
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Scale target
Don't go below 3
Don't go above 15
Keep avg CPU ~70%

## 6. HOW HPA MAKES DECISIONS




Default Formula (CPU):

$$\text{Desired Replicas} = \text{Current Replicas} \times \left( \frac{\text{Current CPU}}{\text{Target CPU}} \right)$$

HPA doesn't change your code.
Its changes your capacity.

## 7. INTERVIEW NOTE

Q: When should you use HPA?

- ☑ When traffic is unpredictable
- ☑ When load varies during the day
- ☑ When CPU/Memory drives scaling
- ☑ When you want cost + performance balance

Q: Can HPA scale to zero?
Not with HPA alone. Use KEDA or custom metrics + HPA for scale to zero.

## 8. TROUBLESHOOTING CHECKLIST

- ☐ Is HPA created and active? `kubectl get hpa`
- ☐ Is metrics-server running?
- ☐ Are metrics available? `kubectl top pods`
- ☐ Are resource requests set?
- ☐ Are min/max replicas reasonable?
- ☐ Check HPA events: `kubectl describe hpa`
- ☐ Is a custom metric available (if used)?

## 9. KEY LESSON

Scaling is not a feature.
It's a survival mechanism.

No HPA = Manual firefighting.
With HPA = Automatic resilience.

AUTOSCALING ISN'T JUST FOR HIGH TRAFFIC APPS.
IT'S FOR ANY APP THAT YOU DON'T WANT TO WAKE UP AT 2 AM AGAIN.

Kubernetes was healthy.
Pods were running.
Traffic was routed.
But the database was the single point of failure.

# Mistake #4: Single Point of Failure Database

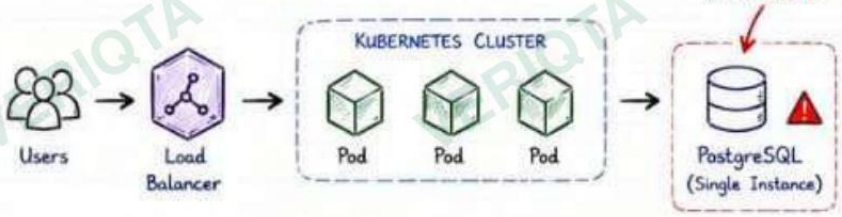
**ONE DATABASE. ONE FAILURE. ENTIRE PLATFORM DOWN.**

## 1. What Happened?

- Our application depended on a single PostgreSQL instance.
- The database hit a failure (disk full + connection exhaustion).
- Connections were dropped.
- All pods started failing at the same time.
- Retries made it worse.
- The entire platform became unavailable.

Kubernetes didn't fail.
The database was the weakest link.

## 2. Current Architecture (The Problem)




All pods look healthy. All nodes look healthy.
But if this database fails, everything fails.

## 3. What Went Wrong?

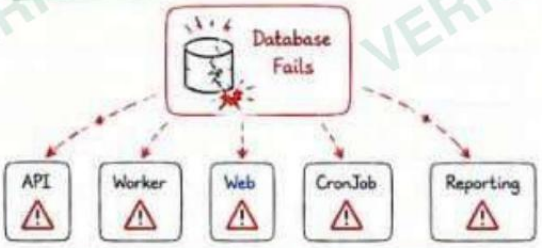
- The database disk filled up.
- New connections were refused.
- Existing connections were reset.
- Application retries exhausted the pool.
- Connection storms amplified the issue.
- Every request failed.
- Users saw timeouts and errors.

ONE FAILURE → SYSTEM WIDE OUTAGE

## 4. The Failure Timeline

| | |
|---|---|
| 02:16 AM | DB connections start rising |
| 02:17 AM | Connection pool exhausted |
| 02:18 AM | DB slow responses |
| 02:19 AM | Errors increase across pods |
| 02:20 AM | Timeouts begin |
| 02:21 AM | All pods failing |
| 02:22 AM | Entire platform down |
| 02:27 AM | DB crashes (disk full) |

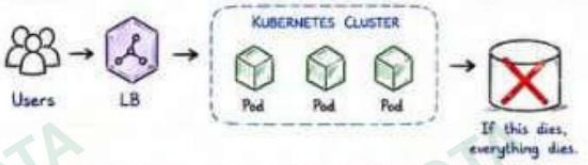
## 5. Blast Radius




- Everything depended on it.
- Everything broke together.

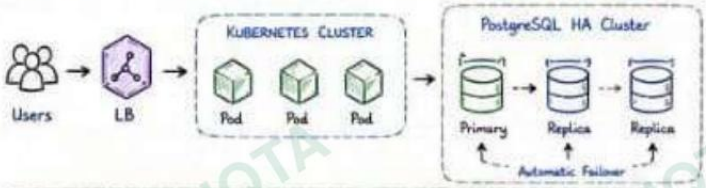
## 6. Single Instance vs High Availability

### Single Instance (What We Had) ☹




- ✗ No redundancy
- ✗ No automatic failover
- ✗ No read scaling
- ✗ Maintenance causes downtime
- ✗ Backups block traffic
- ✗ High risk

### High Availability (What We Should Have Had) ☺




- ✓ Redundancy (replicas)
- ✓ Automatic failover
- ✓ Read scaling
- ✓ Rolling maintenance
- ✓ Backups without downtime
- ✓ High availability

## Bad Signs to Watch For

- → High connection count
- → Increasing wait events
- → Slow queries increasing
- → Disk usage growing
- → Pool timeouts
- → "Too many connections" errors
- → Replication lag increasing

## 8. How We Fixed It

- ✓ Moved to PostgreSQL HA (Primary + 2 Replicas)
- ✓ Added connection pooling (PgBouncer)
- ✓ Set sane connection limits
- ✓ Added read replicas for read-heavy traffic
- ✓ Enabled automated backups
- ✓ Added disk monitoring & alerts
- ✓ Tested failover manually

We removed the single point of failure.
Then we tested it again.

## 9. Troubleshooting Checklist

- ☐ Check DB connectivity (can pods connect?)
- ☐ Check connection count vs max_connections
- ☐ Check long running / blocking queries
- ☐ Check disk usage
- ☐ Check replication lag (if using replicas)
- ☐ Check DB logs for errors
- ☐ Check connection pool settings in the app
- ☐ Check timeouts and retry behavior
- ☐ Check for connection storms

## 10. Key Lesson

If your database is a single point of failure,
your entire platform is a single point of failure.
Design for failure before it fails.

> Kubernetes can make your app resilient.
> Only architecture can make your system resilient.



**REDUNDANCY IS NOT EXPENSIVE. OUTAGES ARE. DESIGN FOR RESILIENCE. SLEEP THROUGH THE NIGHT.**

We couldn't see what was happening. We were debugging in the dark.

# Mistake #5: No Monitoring or Observability Strategy

⚠ WE HAD BLIND SPOTS. BY THE TIME WE KNEW, IT WAS TOO LATE.

## 1. What Happened?

- We had basic Kubernetes metrics.
- But we didn't have application metrics.
- No request tracing.
- No centralized logs.
- No alerts for critical signals.
- We detected the outage from customers, not from our system.

☹ We had data. But no visibility.

## 2. The Blind Spots

| Application Health | User Experience | Dependencies | Resource Pressure | Errors & Exceptions |
|---|---|---|---|---|
| No visibility into real app health or latency. | We didn't know users were timing out. | DB was struggling, but we didn't know why. | CPU looked fine. But we missed memory pressure. | Errors were happening. We didn't see them. |

Monitoring is not just graphs. It's understanding.

## 3. What We Couldn't See (Examples)

- ✕ High latency before timeouts
- ✕ Slow database queries
- ✕ Failed external service calls
- ✕ Memory leaks in the application
- ✕ Error rate increasing
- ✕ Traffic patterns
- ✕ Queue depth / backlog
- ✕ Pods restarting repeatedly


Too many unknowns.

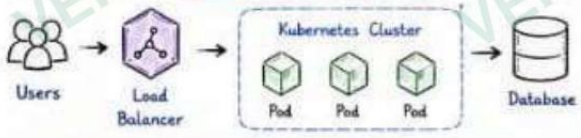
## 4. Observability Pillars (What We Needed)

| Metrics (Tell Us What) | Logs (Tell Us Why) | Traces (Tell Us How) |
|---|---|---|
| CPU, Memory | Centralized Logs | Request Flow |
| RPS, Latency | Structured Logs | Latency Breakdown |
| Errors, Saturation | Searchable | Service Map |
| Custom Metrics | Correlated | Bottlenecks |

☆ Metrics show the symptom. Logs explain it. Traces prove it. Together, they give clarity.

## 5. Before vs After

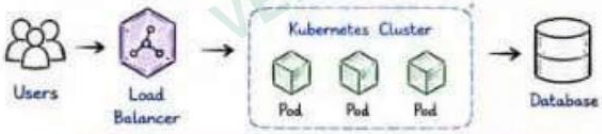
### Before (Blind) ☹




⊗ No dashboards ⊗ No logs
⊗ No alerts ✕ No traces

We were reacting. Not understanding.

### After (Observable) ☺




✓ Dashboards ✓ Tracing
✓ Alerting ✓ SLOs / SLIs
✓ Centralized Logs ✓ Real-time Visibility

We're understanding. We're in control.

## 6. Alerting Examples (What We Should Have Had)

| Alert | Condition | Why It Matters |
|---|---|---|
| High Error Rate | Error rate > 5% for 5m | Users are failing |
| High Latency | P95 latency > 1s for 5m | Users are timing out |
| Pod Restarting | Pod restarts > 3 in 10m | Something is unstable |
| High Memory Usage | Memory > 85% for 5m | Risk of OOM / Eviction |
| DB Connection High | DB connections > 80% | DB may become unavailable |
| Queue Depth High | Queue depth > 1000 | Backlog is building up |

## 7. Quick Wins We Implemented

- ☑ Installed Prometheus + Grafana
- ☑ Added Kubernetes mixin dashboards
- ☑ Added application metrics (RED + USE)
- ☑ Centralized logs with Loki / ELK
- ☐ Added critical alerts (PagerDuty)
- ☐ Configured SLOs for latency & availability
- ☐ Added synthetic checks for user flows
- ☐ Built incident dashboards

Visibility is the first step to stability. ☆

## Interview Note

Q: Why is observability more important than monitoring?

A: Monitoring tells you WHEN something is wrong. Observability helps you understand WHY and HOW it happened.

## 9. Troubleshooting Checklist

- ☐ Do we have dashboards for all critical signals?
- ☐ Do we have alerts for failure conditions?
- ☐ Can we trace a request end-to-end?
- ☐ Are logs centralized and searchable?
- ☐ Do we have SLOs and error budgets?
- ☐ Can we answer "Why is this happening?" fast?

## 10. Key Lesson

You can't fix what you can't see.

Observability turns guesswork into understanding.

OBSERVABILITY IS NOT AN EXPENSE. IT'S AN INSURANCE POLICY FOR YOUR USERS.

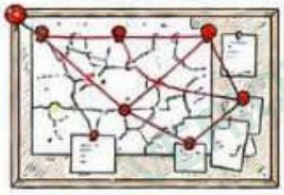
Now that we fixed every issue, let's see how we found them. It wasn't obvious at first.

VERIQTA

☆ Subscriber Series ☆

# The Investigation Timeline

## Connecting the Dots.



⚠ OUTAGES DON'T HAVE A SINGLE CAUSE. THEY HAVE A CHAIN OF CAUSES.

## 1. The Timeline (how we discovered everything)

| 02:16 AM | 02:20 AM | 02:30 AM | 02:55 AM | 03:20 AM | 04:10 AM | 05:00 AM |
|---|---|---|---|---|---|---|
| **Incident Started** | **Page Triggered** | **Initial Triage** | **First Lead Found** | **Deeper Dive** | **Root Causes Identified** | **Fixes Deployed** |
| Users reported timeouts. | On-call received multiple alerts. | Checked pods, services, ingress. Everything looked "normal". | High pod restarts noticed in a few services. | Metrics & logs started revealing the real story. | All 5 issues confirmed. | Stability returned. Monitoring confirmed. |

## 2. What We Found and How We Found It

### 1 No Resource Requests

**Symptoms:**
Some pods pending. Others on overutilized nodes.

**How We Found It:**
"kubectl describe pod" showed scheduling failures.

```
Warning  FailedScheduling
0/6 nodes are available:
Insufficient memory.
```

**Evidence:**
Pods had no requests. Scheduler had no information.

```
resources:
  requests: {}
  limits: {}
```

### 2 Improper Liveness Probes

**Symptoms:**
Pods kept restarting. CrashLoopBackOff increasing.

**How We Found It:**
"kubectl get pods" showed high restart counts.

```
NAME                RESTARTS
web-7d6c8f7b5f-abc  47
api-5c7d9f6f7b-xyz  52
```

**Evidence:**
Probe was too aggressive. Healthy app was being killed.

```
livenessProbe:
  initialDelaySeconds: 5
  periodSeconds: 5
  timeoutSeconds: 1
```

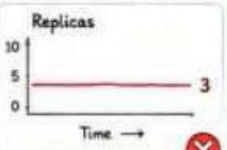
### 3 No Horizontal Pod Autoscaler

**Symptoms:**
CPU high. Queue building. Response time increasing.

**How We Found It:**
Replica count was fixed while traffic spiked.

```
$ kubectl get hpa
No resources found
in default namespace
```

**Evidence:**
No HPA. Replicas stuck at 3.




### 4 Single Point of Failure DB

**Symptoms:**
All services failing despite pods running fine.

**How We Found It:**
Database unreachable errors in app logs.

```
ERROR: could not connect
to server: Connection
refused (SQLSTATE 08001)
```

**Evidence:**
One PostgreSQL instance. No failover. No HA.


### 5 No Monitoring/ Observability

**Symptoms:**
We didn't know any of this until customers told us.

**How We Found It:**
Checked monitoring. Gaps everywhere.


**Evidence:**
No dashboards, no alerts, no traces. We were blind.

## 3. The Dependency Chain (how it all connected)


- Each issue amplified the next one.
- Fixing any single issue wouldn't have solved the outage.

## 4. The Turning Point

At 04:10 AM, we correlated:

- DB errors in all services
- High restarts
- No HPA
- No proper requests
- No observability

→ Then it all made sense.

## 5. What We Learned During Investigation

- ☑ Never trust "everything looks fine" at first glance.
- ☑ Follow the symptoms. They lead to the root cause.
- ☑ Metrics + Logs + Traces = The full picture.
- ☑ Document as you go. It helps during recovery.
- ☑ Blameless investigation = Faster learning.

## Interview Note

**Q:** In what order should you investigate an outage?

**A:** Start from the user impact and move downstream:

User Impact (What) → Services (Where) → Pods (How Many) → Resources → Dependencies

VERIQTA

☆ Subscriber Series

☆ INVESTIGATION ISN'T GUESSWORK. IT'S PATIENCE + LOGIC + GOOD OBSERVABILITY. ☆

VERIQTA

☆ Subscriber Series ☆

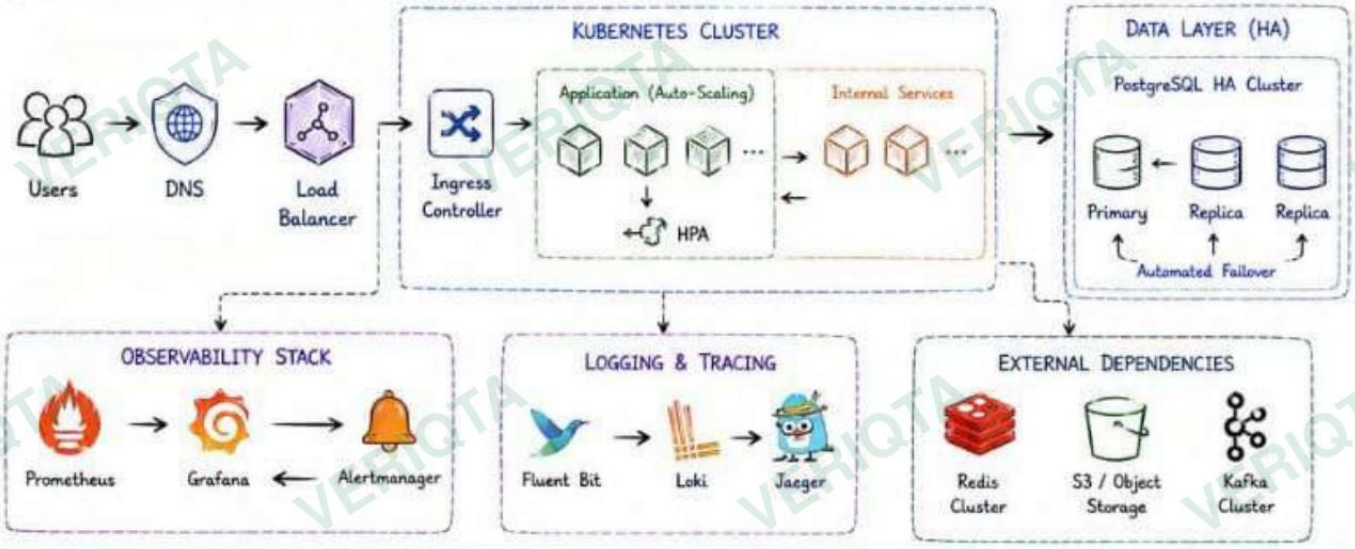
We learned.
We fixed.
We rebuilt it
the right way.

# The Final Production Architecture

## What We Rebuilt. What We Hardened.

✓ Resilient by Design. Observable by Default. Ready for Scale.

## 1. High Level Architecture (Production Ready)



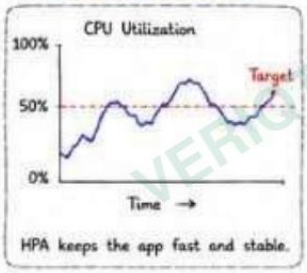

## 2. Compute Layer (Right Sizing + Auto Scaling)

Proper Resource Requests & Limits

```
resources:
  requetes:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1000m"
    memory: "1Gi"
```

✓ Requests for scheduling.
Limits for protection.

Autoscaling That Works




HPA keeps the app fast and stable.

## 3. Reliability Layer (No Single Points of Failure)

| | | |
|---|---|---|
| PostgreSQL HA | → | Primary + Replicas Auto Failover |
| Multi-AZ Nodes | → | Nodes spread across availability zones |
| Redundant Ingress | → | Multiple replicas behind Load Balancer |
| External Dependencies (HA) | → | Redis Cluster, Kafka Cluster, S3 (Durable) |

## 4. Resilience Layer (Self-Healing)

| | | |
|---|---|---|
| Liveness Probes | → | Detects real failures. Restarts only when needed. |
| Readiness Probes | → | Traffic goes only to ready pods. |
| PodDisruptionBudget | → | Keeps minimum pods available during disruptions. |
| Graceful Shutdown | → | Connections drain properly. No dropped requests. |
| Rolling Updates | → | Zero downtime deployments with rollback safety. |

## 5. Observability Layer (See Everything)

| | | |
|---|---|---|
| Metrics (Prometheus) | → | CPU, Memory, RPS, Latency, Errors, Saturation & more. |
| Dashboards (Grafana) | → | Real-time visibility into system health. |
| Logs (Loki) | → | Centralized, searchable, correlated logs. |
| Traces (Jaeger) | → | Request flow, latency breakdown, bottlenecks. |
| Alerts (Alertmanager) | → | Real alerts for real issues. No noise. |

## 6. Security Layer (Secure by Default)

| | | |
|---|---|---|
| Network Policies | → | Only required traffic allowed. |
| RBAC | → | Least privilege access. |
| Secrets Management | → | Secrets stored securely. |
| Image Scanning | → | Vulnerabilities detected early. |
| TLS Everywhere | → | Encrypted in transit. |

## 7. The Outcome

- ☑ Auto scales with traffic
- ☑ Survives failures
- ☑ Fast recovery
- ☑ Full visibility
- ☑ Happy users ☺

This is Producti...

GOOD ARCHITECTURE PREVENTS OUTAGES. | GOOD OBSERVABILITY PREVENTS ...

VERIQTA

☆ Subscriber Series ☆

Use this checklist before your next deployment. Sleep well tonight. Your future self will thank you. ☺

# Production Readiness Checklist

## Don't Learn These in Production.

✓ A resilient system is not an accident. It's a checklist.

### 1. Resource Management

- ☑ CPU & memory requests are set
- ☑ Limits are set and reasonable
- ☑ No containers running without requests
- ☑ Requests match actual usage
- ☑ Use LimitRange & ResourceQuota

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1000m"
    memory: "1Gi"
```

Right sizing = Performance + Stability + Cost Optimization

### 2. Health Checks

- ☑ Liveness probe is configured
- ☑ Readiness probe is configured
- ☑ Probes use correct endpoints
- ☑ Probes have sane timeouts & thresholds
- ☑ Probe failures are tested

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```

Liveness = Is the app alive?
Readiness = Is the app ready?

### 3. Scalability

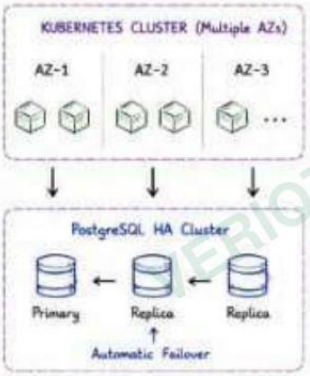
- ☑ HPA is configured for the right metrics
- ☑ Min & Max replicas are set properly
- ☑ Scaling behavior is defined (optional)
- ☑ Load tested to validate scaling
- ☑ Cluster has enough capacity

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 3
  maxReplicas: 15
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Scale proactively. Not reactively.

### 4. High Availability

- ☑ Multi-AZ / Multi-Node deployment
- ☑ No single points of failure
- ☑ Database is HA (Primary + Replicas)
- ☑ Redundant Ingress / Load Balancer
- ☑ Pods spread across nodes




If one zone fails, your users shouldn't notice.

### 5. Observability

- ☑ Multics (Prometheus) is installed
- ☑ Dashboards (Grafana) exist
- ☑ Alerts are configured & tested
- ☑ Logs are centralized
- ☑ Traces are collected (optional)


You can't fix what you can't see. Measure everything.

### 6. Logging

- ☑ Logs are structured (JSON preferred)
- ☑ Centralized logging (Loki / ELK / etc.)
- ☑ Log retention policy is defined
- ☑ Sensitive data is not logged
- ☑ Log search & alerts in place


Logs without structure are just noise.

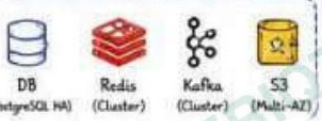
### 7. Security

- ☑ RBAC is enabled and least privilege
- ☑ Secrets are not in code or images
- ☑ NetworkPolicies are enforced
- ☑ Pod Security Standards enabled
- ☑ Images are scanned for vulnerabilities


Security is not a feature. It's a foundation.

### 8. Dependencies

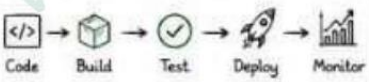
- ☑ External dependencies are HA
- ☑ Timeouts & retries are configured
- ☑ Circuit breaker / bulkhead in place
- ☑ Dependencies are monitored
- ☑ Failover is tested




Your app is only as reliable as its dependencies.

### 9. Deployment & Operations

- ☑ CI/CD pipeline with automated tests
- ☑ Rollouts use rolling update strategy
- ☑ Readiness gates / canary (optional)
- ☑ Rollback strategy is documented
- ☑ Runbooks are up to date




Automate everything you can. Document everything you can't.

### 10. Backups & DR

- ☑ Database backups are automated
- ☑ Backups are encrypted & tested
- ☑ Cross-region backups (if required)
- ☑ Restore process is documented
- ☑ DR plan is tested regularly


Backups you don't test are backups you don't have.

### ☆ Bonus: Pre-Deployment Gate

Before you deploy, answer these:

- → Will this version handle 10x traffic?
- → What happens if a pod / node / AZ fails?
- → Can we detect issues within 1 minute?
- → Can we roll back safely?
- → Will our users notice if something breaks?

If you can't answer "YES" to all above, wait. Improve. Then deploy.

Great teams



CHECKLISTS DON'T SLOW YOU DOWN. THEY SPEED YOU UP BY PREVENTING OUTAGES.

From outage to insight. From chaos to confidence.

☆ Subscriber Series ☆

# 10 Lessons Every Kubernetes Engineer Learns (The Hard Way)

☆ THIS OUTAGE HURT US. THESE LESSONS WILL HELP YOU.

## 1 KUBERNETES IS NOT MAGIC. IT'S RULES + CONFIGURATION.

- Kubernetes does exactly what you tell it.
- Wrong configs = predictable failures.
- Understand the system, don't blindly trust it.

Remember: Know your configs. Own your system.

## 2 RESOURCE REQUESTS ARE NOT OPTIONAL.

- Scheduler makes decisions based on requests.
- No requests = overcommitment.
- Overcommitment = latency, eviction, failures.

Remember: Right requests = right placement.

## 3 BAD PROBES CAN CREATE OUTAGES BY THEMSELVES.

- Liveness kills healthy apps.
- Restart storms take down systems.
- Probes must match real application behavior.

Remember: Probes protect you only if they're right.

## 4 CAPACITY MUST GROW WITH DEMAND.

- Fixed replicas + traffic spikes = overload.
- HPA is not a luxury, it's survival.
- Scale before users feel the pain.

Remember: Design for peak, not average.

## 5 DATABASES ARE OFTEN THE REAL SINGLE POINT OF FAILURE.

- One DB. One failure. Entire platform down.
- HA, replicas, backups — plan for failure.
- Your app is only as strong as your dependencies.

Remember: Remove single points. Build redundancy.

## 6 OBSERVABILITY IS NOT A NICE-TO-HAVE.

- You can't fix what you can't see.
- Metrics tell you WHAT. Logs tell you WHY.
- Traces show you HOW.

Remember: Visibility is the first step to stability.

## 7 MOST OUTAGES ARE CHAINS OF SMALL FAILURES.

- No single issue caused this outage.
- Each small gap amplified the next.
- Fixing one thing would not have saved us.

Remember: Solve the system, not just symptoms.

## 8 ARCHITECTURE MATTERS MORE THAN TOOLS.

- Tools don't make you resilient. Design does.
- Good architecture = boring days.
- Boring systems make happy users.

Remember: Design for failure. Build for resilience.

## 9 AUTOMATE THE REPETITIVE. TEST THE IMPORTANT.

- Automate backups, rollbacks, and alerts.
- Test failover, restores, and scaling.
- If you don't test it, it's not real.

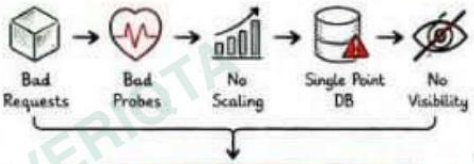
Remember: Practice failure so it's not a surprise.

## 10 RESILIENCE IS A JOURNEY, NOT A DESTINATION.

- Systems change. Traffic changes. Risks change.
- Keep improving. Keep learning. Keep hardening.
- Today's fix is tomorrow's baseline.

Remember: Continuous improvement is resilience.

### THE OUTAGE IN ONE PICTURE




* We fixed the chain. We built a better system. Now we sleep better. Our users do too.

### SUCCESS LOOKS LIKE THIS

- ☑ Users happy
- ☑ Systems stable
- ☑ Alerts are quiet (by design)
- ☑ Teams move fast
- ☑ We learn and improve

That's the goal.
That's production excellence.

### TO EVERY ENGINEER READING THIS

You will face outages.
It's not a matter of if, but when.

What matters is:

- ☑ How fast you learn
- ☑ How well you build
- ☑ How quickly you recover

Keep learning.
Keep building.
Keep shipping.

"WE DON'T AIM FOR PERFECTION. WE AIM FOR RESILIENCE."

Thank you for readi
Stay resilient. Stay
See you in the nex

VERIQTA ☆ Subscriber Series

BUILD RESILIENT SYSTEMS. DELIVER VALUE. MAKE USERS HAPPY.